

Finite Volume Solver for 2D Compressible Flow using CuPy – CUDA

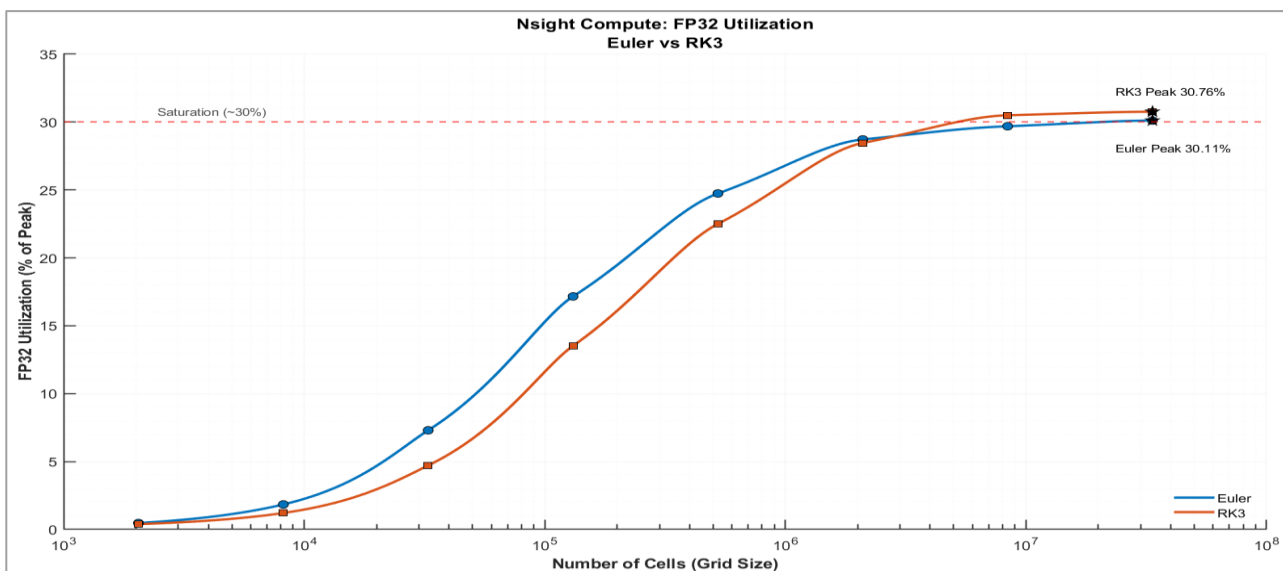
Tanish Shetty - 251010052

Specs of Code1 on RTX 6000:

- **Programming Stack:** Python (CuPy) + CUDA C (JIT-compiled kernels via CuPy RawModule)
- **Flux Scheme:** Rusanov - chosen for low computational cost and GPU efficiency
- **Time Integration:**
 - Euler (single-stage, memory-efficient)
 - RK3 (three-stage, higher arithmetic intensity with increased memory traffic)
- **Grid Sizes:** 128×16, 256×32, 512×64, 1024×128, 2048×256, 4096×512, 8192×1024, 16384×2048.
- **Parallelization Strategy:**
 - 2D thread mapping (one thread per control volume)
 - Uniform workload and high occupancy
 - Shared memory tiling for stencil reuse
 - Coalesced global memory access
 - Persistent thread blocks to reduce kernel launch overhead
- **Performance Design Goal:**
 - Optimize for **memory-bound execution regime**
 - Prioritize **data locality and bandwidth efficiency over increased arithmetic operations**

FLOPS rating: Performance evaluated using **Nsight Compute hardware FP32 utilization metric:**

- Peak FP32 Throughput: Both schemes (Euler and RK3) converge to ~30% of peak FP32 performance at (16384×2048), reflecting saturation of hardware utilization under memory-bound constraints.

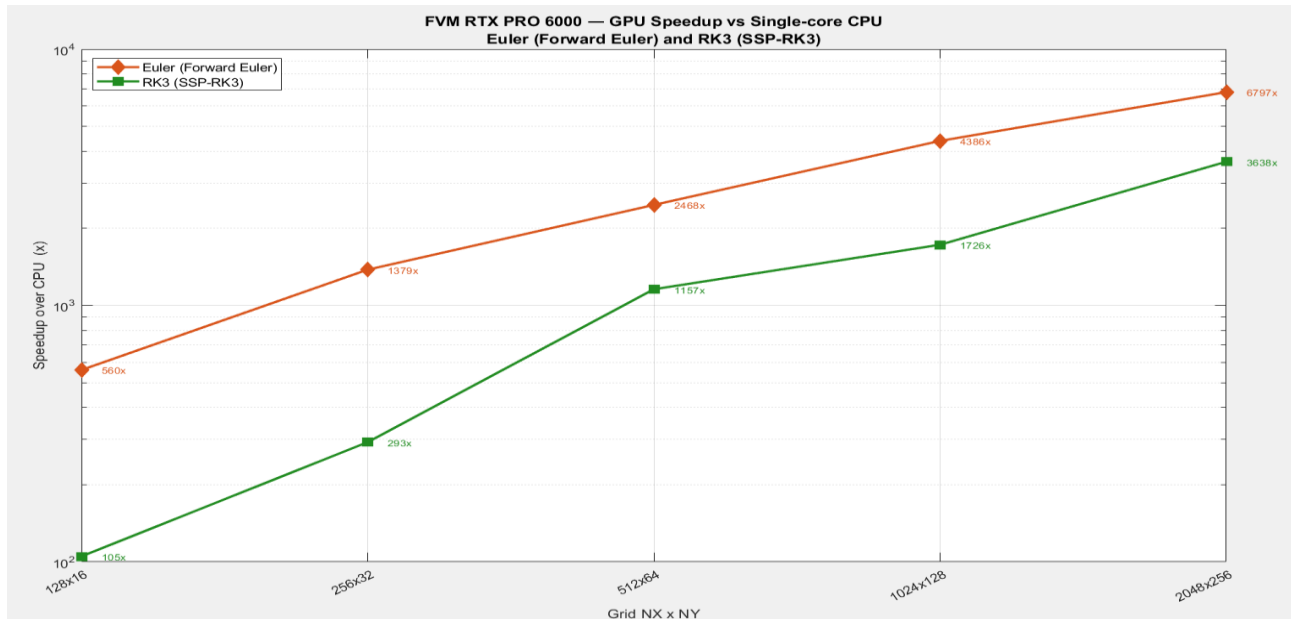


Nsight highlights:

- Hardware efficiency improves significantly with grid size, with SM utilization increasing from ~1% to ~78%, reducing launch overhead effects
- L1 utilization increases significantly: ~0.70% → ~49%. Confirms **tiling + temporal blocking are working as intended.**
- Active warps stabilize at ~60–66%, indicating near-optimal occupancy for latency hiding under current register constraints.

- DRAM throughput peaks at $\sim 50\%$ (RK3) while SM utilization reaches $\sim 75\text{--}79\%$, indicating performance is limited by memory latency rather than raw bandwidth saturation

Speedup compared to CPU:



Best things of your code & project:

- Majority of computations are fused within a minimal number of kernel launches, reducing global memory round-trips and keeping data in on-chip memory (registers/L1), improving effective bandwidth utilization.
- Multi-stage updates are performed with data reuse across stages, avoiding redundant global loads and increasing arithmetic intensity.
- High register usage ($\sim 80/\text{thread}$) trades occupancy for reduced memory access, achieving better overall throughput.
- Robustness across regimes: Handles both smooth regions (high-order accuracy) and discontinuities (stable shock capture) within the same framework.

Figures:

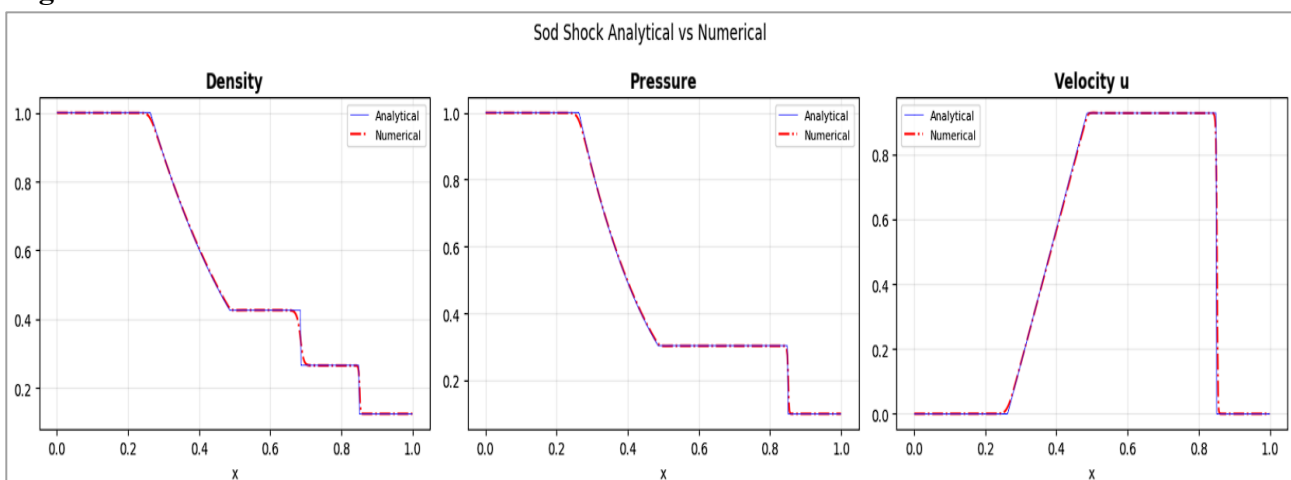


Figure: Validation of the FVM–MUSCL–HLL solver

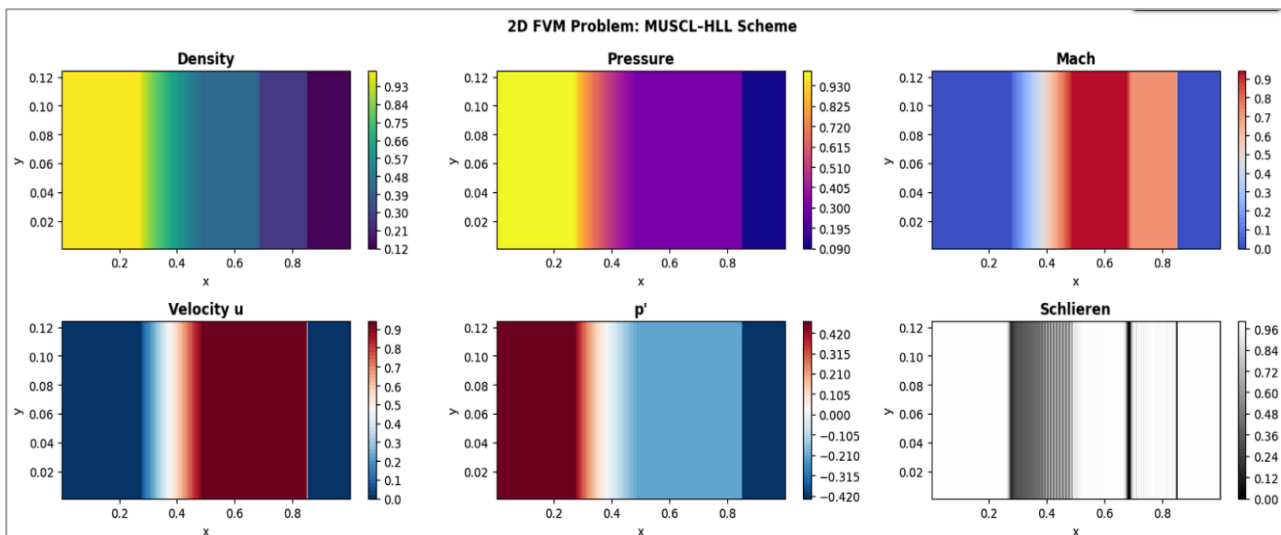


Figure: 2D Flow Field Visualization of Sod Shock Tube using FVM-MUSCL-HLL Scheme

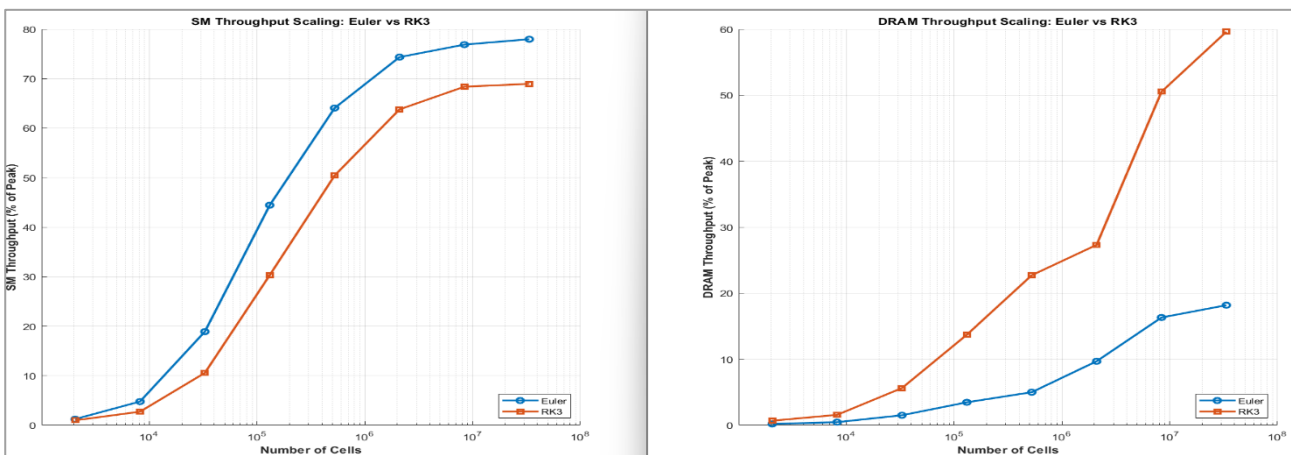


Figure: SM and DRAM Throughput Scaling with Grid Size: Euler vs RK3

Bottlenecks:

- Built-in Euler vs RK3 comparison
Directly links numerical accuracy with GPU performance and memory behavior
- Memory latency-dominated execution: High SM utilization (~ 75 – 79%) but low FP32 ($\sim 30\%$) indicates warp stalls due to memory access delays.
- Performance saturates beyond $\sim O(10^6)$ cells as hardware utilization approaches steady state.
- ~ 80 registers/thread caps occupancy at $\sim 50\%$, directly limiting active warps/SM and reducing the GPU's ability to hide memory latency